

The idea was that for a single user and a single e-mail, extra computational effort was not noticeable, but someone sending a large number of spam e-mails would be discouraged as the time and resources required to run the spam campaign would increase substantially.

B-money was proposed by *Wei Dai* in 1998, which introduced the idea of using Proof of Work to create money. A major weakness in the system was that an adversary with higher computational power could generate unsolicited money without allowing the network to adjust to an appropriate difficulty level. The system lacked details on the consensus mechanism between nodes and some security issues such as Sybil attacks were also not addressed. At the same time, *Nick Szabo* introduced the concept of BitGold, which was also based on the Proof of Work mechanism but had the same problems as b-money with the exception that the network difficulty level was adjustable. *Tomas Sander* and *Ammon TaShama* introduced an e-cash scheme in 1999 that, for the first time, used Merkle trees to represent coins and zero knowledge proofs to prove the possession of coins. In the scheme, a central bank was required that kept a record of all used serial numbers. This scheme allowed users to be fully anonymous albeit at a computational cost. **RPOW (Reusable Proof of Work)** was introduced by *Hal Finney* in 2004 and used the hashcash scheme by *Adam Back* as a proof of computational resources spent to create the money. This was also a central system that kept a central database to keep track of all used POW tokens. This was an online system that used remote attestation made possible by a **trusted computing platform (TPM hardware)**.

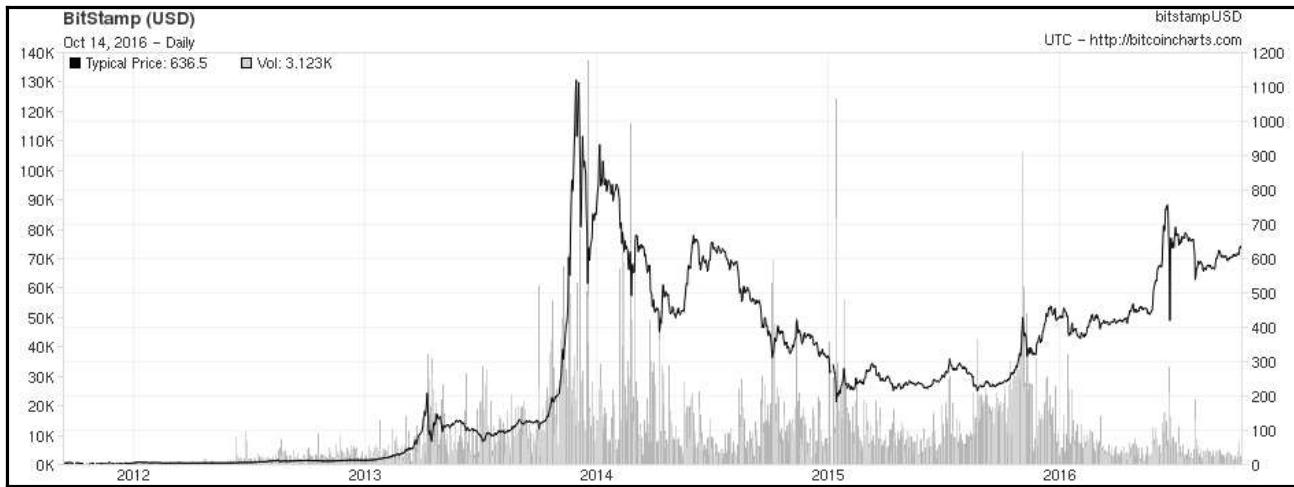
All the previously mentioned schemes are intelligently designed but were weak from one aspect or another. Especially, all these schemes rely on a central server that is required to be trusted by the users.

Bitcoin

In 2008, a paper on bitcoin, *Bitcoin: A Peer-to-Peer Electronic Cash System* was written by *Satoshi Nakamoto*. The first key idea introduced in the paper was that purely peer-to-peer electronic cash that does need an intermediary bank to transfer payments between peers.

Bitcoin is built on decades of cryptographic research such as the research in Merkle trees, hash functions, public key cryptography, and digital signatures. Moreover, ideas such as BitGold, b-money, hashcash, and cryptographic time stamping provided the foundations for bitcoin invention. All these technologies are cleverly combined in bitcoin to create the world's first decentralized currency. The key issue that has been addressed in bitcoin is an elegant solution to the Byzantine Generals problem along with a practical solution of the double-spend problem.

The value of bitcoin has increased significantly since 2011, as shown in the following graph:



Bitcoin price and volume since 2012 (on logarithmic scale)

The regulation of bitcoin is a controversial subject and as much as it is a libertarian's dream, law enforcement agencies and governments are proposing various regulations to control it, such as BitLicense issued by New York's state department of financial services. This is a license issued to businesses that perform activities related to virtual currencies.

The growth of Bitcoin is also due to so-called *Network Effect*. Also called demand-side economies of scale, it is a concept that basically means more users who use the network, the more valuable it becomes. Over time, an exponential increase has been seen in the Bitcoin network growth. Even though the price of bitcoin is quite volatile, it has increased significantly over the last few years. Currently (at the time of writing this), bitcoin price is 815 GBP.

Bitcoin definition

Bitcoin can be defined in various ways; it's a protocol, a digital currency, and a platform. It is a combination of peer-to-peer network, protocols, and software that facilitate the creation and usage of the digital currency named bitcoin. Note that Bitcoin with a capital *B* is used to refer to the Bitcoin protocol, whereas bitcoin with a lowercase *b* is used to refer to bitcoin, the currency. Nodes in this peer-to-peer network talk to each other using the Bitcoin protocol.

Decentralization of currency was made possible for the first time with the invention of bitcoin. Moreover, the double spending problem was solved in an elegant and ingenious way in bitcoin. Double spending problem arises when, for example, a user sends coins to two different users at the same time and they are verified independently as valid transactions.

Keys and addresses

Elliptic curve cryptography is used to generate public and private key pairs in the Bitcoin network. The bitcoin address is created by taking the corresponding public key of a private key and hashing it twice, first with the SHA256 algorithm and then with RIPEMD160. The resultant 160-bit hash is then prefixed with a version number and finally encoded with a Base58Check encoding scheme. The bitcoin addresses are 26-35 characters long and begin with digit 1 or 3. A typical bitcoin address looks like a string shown here:

1ANAgUGG8bikEv2fYsTBnRUmx7QUcK58wt

This is also commonly encoded in a QR code for easy sharing. The QR code of the preceding address is shown in the following image:



QR code of a bitcoin address 1ANAgUGG8bikEv2fYsTBnRUmx7QUcK58wt

Currently, there are two types of addresses, the commonly used P2PKH and another P2SH type, starting with 1 and 3, respectively. In the early days, bitcoin used direct Pay-to-Pubkey, which is now superseded by P2PKH. However, direct Pay-to-Pubkey is still used in bitcoin for coinbase addresses. Addresses should not be used more than once; otherwise, privacy and security issues can arise. Avoiding address reuse circumvents anonymity issues to an extent, bitcoin has some other security issues as well, such as transaction malleability, which requires different approaches to resolve.



From bitaddress.org private key and bitcoin address in a paper wallet

Public keys in bitcoin

In public key cryptography, public keys are generated from private keys. Bitcoin uses ECC based on the SECP256K1 standard. A private key is randomly selected and is 256-bit in length. Public keys can be presented in an uncompressed or compressed format. Public keys are basically x and y coordinates on an elliptic curve and in an uncompressed format and are presented with a prefix of 04 in a hexadecimal format. X and Y coordinates are both 32-bit in length. In total, the compressed public key is 33 bytes long as compared to 65 bytes in the uncompressed format. The compressed version of public keys basically includes only the X part, since the Y part can be derived from it. The reason why the compressed version of public keys works is that the bitcoin client initially used uncompressed keys, but starting from bitcoin core client 0.6, compressed keys are used as the standard.

Keys are identified by various prefixes, described as follows:

- Uncompressed public keys used 0x04 as the prefix
- Compressed public key starts with 0x03 if the y 32-bit part of the public key is odd
- Compressed public key starts with 0x02 if the y 32-bit part of the public key is even

The more detailed mathematical description and the reason why it works is described here. If the ECC graph is visualized, it reveals that the y coordinate can be either below the x axis or above the x axis and as the curve is symmetric, only the location in the prime field is required to be stored.

Private keys in bitcoin

Private keys are basically 256-bit numbers chosen in the range specified by the SECP256K1 ECDSA recommendation. Any randomly chosen 256-bit number from 0x1 to 0xFFFF FFFF FFFF FFFF FFFF FFFF BAAE DCE6 AF48 A03B BFD2 5E8C D036 4140 is a valid private key.

Private keys are usually encoded using **Wallet Import Format (WIF)** in order to make them easier to copy and use. WIF can be converted into private key and vice versa. The steps are described here.

Also, **Mini Private Key Format** is sometimes used to encode the key in under 30 characters in order to allow storage where physical space is limited, for example, etching on physical coins or damage-resistant QR codes. The bitcoin core client also allows the encryption of the wallet that contains the private keys.

Bitcoin currency units

Bitcoin currency units are described as follows. The smallest bitcoin denomination is the Satoshi.

DENOMINATION	↕ ABBREVIATION	↕ FAMILIAR NAME	↕ VALUE IN BTC
Satoshi	SAT	Satoshi	0.00000001 BTC
Microbit	μBTC (uBTC)	Microbitcoin or Bit	0.000001 BTC
Millibit	mBTC	Millibitcoin	0.001 BTC
Centibit	cBTC	Centibitcoin	0.01 BTC
Decibit	dBTC	Decibitcoin	0.1 BTC
Bitcoin	BTC	Bitcoin	1 BTC
DecaBit	daBTC	Decabitcoin	10 BTC
Hectobit	hBTC	Hectobitcoin	100 BTC
Kilobit	kBTC	Kilobitcoin	1000 BTC
Megabit	MBTC	Megabitcoin	1000000 BTC

Base58Check encoding

This encoding is used to limit the confusion between various characters, such as 0OI1 as they can look the same in different fonts. The encoding basically takes the binary byte arrays and converts them into human-readable strings. This string is composed by utilizing a set of 58 alphanumeric symbols. More explanation and logic can be found in the `base58.h` source file in the bitcoin source code.

```
6  /**
7   * Why base-58 instead of standard base-64 encoding?
8   * - Don't want 0OI1 characters that look the same in some fonts and
9   *   could be used to create visually identical looking data.
10  * - A string with non-alphanumeric characters is not as easily accepted as input.
11  * - E-mail usually won't line-break if there's no punctuation to break at.
12  * - Double-clicking selects the whole string as one word if it's all alphanumeric.
13  */
14  #ifndef BITCOIN_BASE58_H
```

Explanation from the bitcoin source code

Bitcoin addresses are encoded using the Base58check encoding.

Vanity addresses

As bitcoin addresses are based on base 58 encoding, it is possible to generate addresses that contain human-readable messages. An example is shown as follows:



Public address encoded in QR

Vanity addresses are generated using a purely brute-force method. An example is shown in the following screenshot:



Vanity address generated from <https://bitcoinvanitygen.com/>

Transactions

Transactions are at the core of the bitcoin ecosystem. Transactions can be as simple as just sending some bitcoins to a bitcoin address, or it can be quite complex depending on the requirements. Each transaction is composed of at least one input and output. Inputs can be thought of as coins being spent that have been created in a previous transaction and outputs as coins being created. If a transaction is minting new coins, then there is no input and therefore no signature is needed. If a transaction is to send coins to some other user (a bitcoin address), then it needs to be signed by the sender with their private key and a reference is also required to the previous transaction in order to show the origin of the coins. Coins are, in fact, unspent transaction outputs represented in Satoshis.

Transactions are not encrypted and are publicly visible in the blockchain. Blocks are made up of transactions and these can be viewed using any online blockchain explorer.

The transaction life cycle

1. A user/sender sends a transaction using wallet software or some other interface.

2. The wallet software signs the transaction using the sender's private key.
3. The transaction is broadcasted to the Bitcoin network using a flooding algorithm.
4. Mining nodes include this transaction in the next block to be mined.
5. Mining starts once a miner who solves the Proof of Work problem broadcasts the newly mined block to the network. Proof of Work is explained in detail later in this chapter.
6. The nodes verify the block and propagate the block further, and confirmation starts to generate.
7. Finally, the confirmations start to appear in the receiver's wallet and after approximately six confirmations, the transaction is considered finalized and confirmed. However, six is just a recommended number; the transaction can be considered final even after the first confirmation. The key idea behind waiting for six confirmations is that the probability of double spending is virtually eliminated after six confirmations.

The transaction structure

A transaction at a high level contains metadata, inputs, and outputs. Transactions are combined to create a block. The transaction structure is shown in the following table:

Field	Size	Description
Version Number	4 bytes	Used to specify rules to be used by the miners and nodes for transaction processing.
Input counter	1 bytes – 9 bytes	The number of inputs included in the transaction.
list of inputs	variable	Each input is composed of several fields, including Previous transaction hash, Previous Txout-index, Txin-script length, Txin-script, and optional sequence number. The first transaction in a block is also called a coinbase transaction. It specifies one or more transaction inputs.
Out-counter	1 bytes – 9 bytes	A positive integer representing the number of outputs.
list of outputs	variable	Outputs included in the transaction.
lock_time	4 bytes	This defines the earliest time when a transaction becomes valid. It is either a Unix timestamp or a block number.

- **MetaData:** This part of the transaction contains some values such as the size of the transaction, the number of inputs and outputs, the hash of the transaction, and a `lock_time` field. Every transaction has a prefix specifying the version number.
- **Inputs:** Generally, each input spends a previous output. Each output is considered an **Unspent Transaction Output (UTXO)** until an input consumes it.
- **Outputs:** Outputs have only two fields, and they contain instructions for the sending of bitcoins. The first field contains the amount of Satoshis, whereas the second field is a locking script that contains the conditions that need to be met in order for the output to be spent. More information on transaction spending using locking and unlocking scripts and producing outputs is discussed later in this section.
- **Verification:** Verification is performed using bitcoin's scripting language.

A sample transaction is shown as follows:

```
{
  "txid": "08af7960ca9255c67686296fb65452ed3f96f18831c9a3d8ea552e4ccee5c4af",
  "hash": "08af7960ca9255c67686296fb65452ed3f96f18831c9a3d8ea552e4ccee5c4af",
  "size": 226,
  "vsize": 226,
  "version": 1,
  "locktime": 969523,
  "vin": [
    {
      "txid": "3e553260a0a94860f7043eb6576e15e6cfeb2990aea961210ae1fde328bb08b0",
      "vout": 1,
      "scriptSig": {
        "asm":
"3045022100cfb31edabc62c82b41d12f651d2e3e013ee1a7ee2bb4526f3dda640e6d8d224502207d8d1d8e41350b9cdf36f389f942ab68c12f113fe99014f5d6df6610407877d2[ALL] 037bc82d0078993f6943e7ff6e82e82da600f34edc8bca136331a9901c8bb60b0d",
        "hex":
"483045022100cfb31edabc62c82b41d12f651d2e3e013ee1a7ee2bb4526f3dda640e6d8d224502207d8d1d8e41350b9cdf36f389f942ab68c12f113fe99014f5d6df6610407877d20121037bc82d0078993f6943e7ff6e82e82da600f34edc8bca136331a9901c8bb60b0d"
      },
      "sequence": 4294967294
    }
  ],
  "vout": [
    {
      "value": 2.30000000,
      "n": 0,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 07e78644a61343068fa8d4940a79976e758ac6ef OP_EQUALVERIFY OP_CHECKSIG",
        "hex": "76a91407e78644a61343068fa8d4940a79976e758ac6ef88ac",
        "reqSigs": 1,
        "type": "pubkeyhash",
        "addresses": [
          "mgEkNzxV3qbYtDKEKTvo1VpgJ63Au619q2"
        ]
      }
    }
  ]
}
```

A sample decoded transaction, showing various fields described earlier

The script language

Bitcoin uses a simple stack-based language called *script* to describe how bitcoins can be spent and transferred. It is not Turing complete and has no loops to avoid any undesirable effects of long running/hung scripts on the bitcoin network. This scripting language is based on a Forth-like syntax and uses a reverse polish notation in which every operand is followed by its operators. It is evaluated from the left to the right using a **Last in First Out (LIFO)** stack.

Scripts use various Opcodes or instructions to define their operation. Opcodes are also known as words, commands, or functions. Earlier versions of the bitcoin node had a few Opcodes that are no longer used due to bugs discovered in their design.

The various categories of the scripting Opcodes are constants, flow control, stack, bitwise logic, splice, and arithmetic, cryptography, and lock time.

A transaction script is evaluated by combining `ScriptSig` and `ScriptPubKey`. `ScriptSig` is the unlocking script, whereas `ScriptPubKey` is the locking script. This is how a transaction is evaluated to be spent; first, it is unlocked and then it is spent. `ScriptSig` is provided by the user who wishes to unlock the transaction. `ScriptPubkey` is part of the transaction output and specifies the conditions that need to be fulfilled in order to spend the output. In other words, outputs are locked by the `ScriptPubKey` (Locking script) that contains the conditions, when met will unlock the output, and coins can then be redeemed.

Commonly used Opcodes

All Opcodes are declared in the `script.h` file in the bitcoin reference client source code. This can be accessed from the link at <https://github.com/bitcoin/bitcoin/blob/master/src/script/script.h> under the following comment:

```
/** Script opcodes */
```

A description of the most commonly used Opcodes is listed here. This table is taken from the bitcoin developer's guide:

Opcode	Description
OP_CHECKSIG	This takes a public key and signature and validates the signature of the hash of the transaction. If it matches, then TRUE is pushed onto the stack; otherwise, FALSE is pushed.
OP_EQUAL	This returns 1 if the inputs are exactly equal; otherwise, 0 is returned.
OP_DUP	This duplicates the top item in the stack.
OP_HASH160	The input is hashed twice, first with SHA-256 and then with RIPEMD-160.
OP_VERIFY	This marks the transaction as invalid if the top stack value is not true.
OP_EQUALVERIFY	This is the same as OP_EQUAL, but it runs OP_VERIFY afterwards.
OP_CHECKMULTISIG	This takes the first signature and compares it against each public key until a match is found and repeats this process until all signatures are checked. If all signatures turn out to be valid, then a value of 1 is returned as a result; otherwise, 0 is returned.

Types of transaction

There are various scripts available in bitcoin to handle the value transfer from the source to the destination. These scripts range from very simple to quite complex depending upon the requirements of the transaction. Standard transaction types are discussed here. Standard transactions are evaluated using `IsStandard()` and `IsStandardTx()` tests and only standard transactions that pass the test are generally allowed to be mined or broadcasted on the bitcoin network. However, nonstandard transactions are valid and allowed on the network.

- **Pay to Public Key Hash (P2PKH):** P2PKH is the most commonly used transaction type and is used to send transactions to the bitcoin addresses. The format of the transaction is shown as follows:

```
ScriptPubKey: OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY  
OP_CHECKSIG
```

```
ScriptSig: <sig> <pubKey>
```

13. Using the referenced output transactions to get input values, verify that each input value, as well as the sum, is in the allowed range of 0-21 million BTC.
14. Reject the transaction if the sum of input values is less than the sum of output values.
15. Reject the transaction if the transaction fee would be too low to get into an empty block.

Blockchain

Blockchain is a public ledger of a timestamped, ordered, and immutable list of all transactions on the bitcoin network. Each block is identified by a hash in the chain and is linked to its previous block by referencing the previous block's hash.

In the following structure of a block, a block header is described, followed by a detailed diagram that provides an insight into the blockchain structure.

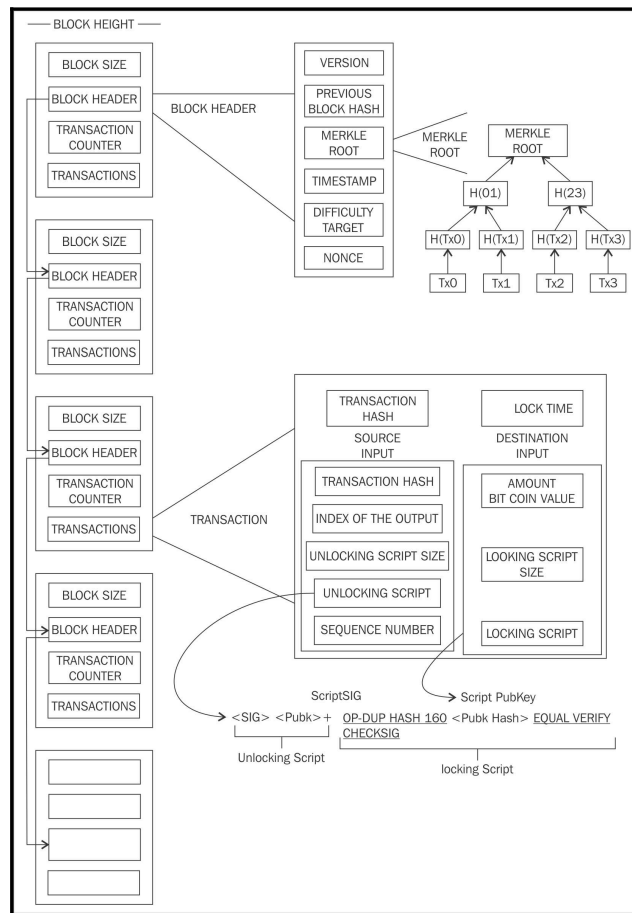
The structure of a block

Bytes	Name	Description
80	Block header	This includes fields from the block header described in the next section.
<i>variable</i>	Transaction counter	The field contains the total number of transactions in the block, including the coinbase transaction.
<i>variable</i>	Transactions	All transactions in the block.

The structure of a block header

Bytes	Name	Description
4	Version	The block version number that dictates the block validation rules to follow.
32	previous block header hash	This is a double SHA256 hash of the previous block's header.
32	merkle root hash	This is a double SHA256 hash of the merkle tree of all transactions included in the block.

4	Timestamp	This field contains the approximate creation time of the block in the Unix epoch time format. More precisely, this is the time when the miner has started hashing the header (the time from the miner's point of view).
4	Difficulty target	This is the difficulty target of the block.
4	Nonce	This is an arbitrary number that miners change repeatedly in order to produce a hash that fulfills the difficulty target threshold.



A visualization of blockchain, block, block header, transaction and script

As shown in the preceding diagram, blockchain is a chain of blocks where each block is linked to its previous block by referencing the previous block header's hash. This linking makes sure that no transaction can be modified unless the block that records it and all blocks that follow it are also modified. The first block is not linked to any previous block and is known as the genesis block.

The genesis block

This is the first block in the bitcoin blockchain. The genesis block was hardcoded in the bitcoin core software. It is in the `chainparams.cpp` file.

<https://github.com/bitcoin/bitcoin/blob/master/src/chainparams.cpp>

```
48  * CtxOut(nValue=50.00000000, scriptPubkey=0x5f10f1b028704c8a578d08)
49  * vMerkleTree: 4a5e1e
50  */
51  static CBlock CreateGenesisBlock(uint32_t nTime, uint32_t nNonce, uint32_t nBits, int32_t nVersion, const CAmount& genesisReward)
52  {
53      const char* pszTimestamp = "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks";
54      const CScript genesisOutputScript = CScript() << ParseHex("04678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649");
55      return CreateGenesisBlock(pszTimestamp, genesisOutputScript, nTime, nNonce, nBits, nVersion, genesisReward);
56  }
57
58  /**
```

Bitcoin provides protection against double spending by enforcing strict rules on transaction verification and via mining. Blocks are added in the blockchain only after strict rule checking and successful Proof of Work solution. Block height is the number of blocks before a particular block in the blockchain. The current height (at the time of writing this) of the blockchain is 434755 blocks. Proof of Work is used to secure the blockchain. Each block contains one or more transactions, out of which the first transaction is a coinbase transaction. There is a special condition for coinbase transactions that prevent them to be spent until at least 100 blocks in order to avoid a situation where the block may be declared stale later on.

Stale blocks are created when a block is solved and every other miner who is still working to find a solution to the hash puzzle is working on that block. Mining and hash puzzles will be discussed later in the chapter in detail. As the block is no longer required to be worked on, this is considered a stale block.

Orphan blocks are also called detached blocks and were accepted at one point in time by the network as valid blocks but were rejected when a proven longer chain was created that did not include this initially accepted block. They are not part of the main chain and can occur at times when two miners manage to produce the blocks at the same time.

The latest block version is version 4, which was proposed with BIP65 and has been used since bitcoin core client 0.11.2 since the implementation of BIP9 bits in `nVersion` field are being used to indicate softfork changes.

These filters are mainly used by simple payment verification SPV clients to request transactions and the merkle blocks they are interested in. A merkle block is a lightweight version of the block, which includes a block header, some hashes, a list of 1-bit flags, and a transaction count. This information can then be used to build a merkle tree. This is achieved by creating a filter that matches only those transaction and blocks that have been requested by the SPV client. Once version messages have been exchanged and connection has been established between peers, the nodes can set filters according to their requirements. These probabilistic filters offer a varying degree of privacy or precision depending upon how accurately or loosely they have been set. A strict bloom filter will only filter transactions that have been requested by the node but at the expense of the possibility of revealing the user addresses to adversaries who can correlate transactions with their IP addresses, thus compromising privacy. On the other hand, a loosely set filter can result in retrieving more unrelated transactions but will offer more privacy. Also, for SPV clients, bloom filters allow them to use low bandwidth as opposed to downloading all transactions for verification.

- **BIP 37** proposed the bitcoin implementation of bloom filters and introduced three new messages to the Bitcoin protocol.
- **Filterload:** This is used to set the bloom filter on the connection.
- **Filteradd:** This adds a new data element to the current filter.
- **FilterClear:** This deletes the currently loaded filter.

More details can be found in the BIP37 specification.

Wallets

The wallet software is used to store private or public keys and bitcoin address. It performs various functions, such as receiving and sending bitcoins. Nowadays, software usually offers both functionalities: bitcoin client and wallet. On the disk, the bitcoin core client wallets are stored as the Berkeley DB file:

```
: ~/.bitcoin$ file wallet.dat
```

wallet.dat: Berkeley DB (Btree, version 9, native byte-order)

Private keys can be generated in different ways and are used by different types of wallets. Wallets do not store any coins, and there is no concept of wallets storing balance or coins for a user. In fact, in the bitcoin network, *coins* do not exist; instead, only transaction information is stored on the blockchain (more precisely, UTXO, unspent outputs), which are then used to calculate the amount of bitcoins.

Wallet types

In bitcoin, there are different types of wallets that can be used to store private keys. As a software program, they also provide some functions to the users to manage and carry out transactions on the bitcoin network.

Non-deterministic wallets

These wallets contain randomly generated private keys and are also called *Just a Bunch of Key wallets*. The bitcoin core client generates some keys when first started and generates keys as and when required. Managing a large number of keys is very difficult and an error-prone process can lead to theft and loss of coins. Moreover, there is a need to create regular backups of the keys and protect them appropriately in order to prevent theft or loss.

Deterministic wallets

In this type of wallet, keys are derived out of a seed value via hash functions. This seed number is generated randomly and is commonly represented by human-readable *mnemonic code* words. Mnemonic code words are defined in BIP39. This phrase can be used to recover all keys and makes private key management comparatively easier.

Hierarchical deterministic wallets

Defined in BIP32 and BIP44, HD wallets store keys in a tree structure derived from a seed. The seed generates the parent key (master key), which is used to generate child keys and, subsequently, grandchild keys. Key generation in HD wallets does not generate keys directly; instead, it produces some information (private key generation information) that can be used to generate a sequence of private keys. The complete hierarchy of private keys in an HD wallet is easily recoverable if the master private key is known. It is because of this property that HD wallets are very easy to maintain and are highly portable.

Brain wallets

The master private key can also be derived from the hash of passwords that are memorized. The key idea is that this passphrase is used to derive the private key and if used in HD wallets, this can result in a full HD wallet that is derived from a single memorized password. This is known as brain wallet. This method is prone to password guessing and brute force attacks but techniques such as *key stretching* can be used to slow down the progress made by the attacker.

Paper wallets

As the name implies, this is a paper-based wallet with the required key material printed on it. It requires physical security to be stored. Paper wallets can be generated online from various service providers, such as <https://bitcoinpaperwallet.com/> or <https://www.bitaddress.org/>.

Hardware wallets

Another method is to use a tamper-resistant device to store keys. This tamper-resistant device can be custom-built or with the advent of NFC-enabled phones, this can also be a **secure element (SE)** in NFC phones. Trezor and Ledger wallets (various types) are the most commonly used bitcoin hardware wallets.



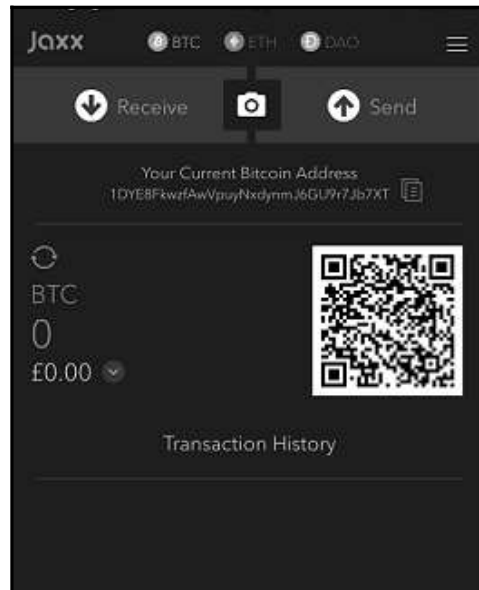
Trezor Wallet

Online wallets

Online wallets, as the name implies, are stored entirely online and are provided as a service usually via cloud. They provide a web interface to the users to manage their wallets and perform various functions such as making and receiving payments. They are easy to use but imply that the user trust the online wallet service provider.

Mobile wallets

Mobile wallets, as the name suggests, are installed on mobile devices. They can provide various methods to make payments, most notably the ability to use smart phone cameras to scan QR codes quickly and make payments. Mobile wallets are available for the Android platform and iOS, for example, breadwallet, copay, and Jaxx.



Jaxx Mobile wallet

Bitcoin payments

Bitcoins can be accepted as payments using various techniques. Bitcoin is not recognized as a legal currency in many jurisdictions, but it is increasingly being accepted as a payment method by many online merchants and e-commerce websites. There are a numbers of ways in which buyers can pay the business that accepts bitcoins. For example, in an online shop, bitcoin merchant solutions can be used, whereas in traditional physical shops, point of sale terminals and other specialized hardware can be used. Customers can simply scan the QR barcode with the seller's payment URI in it and pay using their mobile devices. Bitcoin URIs allow users to make payments by simply clicking on links or scanning QR codes. **URI (Uniform Resource Identifier)** is basically a string that represents the transaction information. It is defined in BIP21. The QR code can be displayed near the point of the sale terminal. Nearly all bitcoin wallets support this feature.

6

Smart Contracts

This chapter provides an introduction to smart contracts. This is not a new concept, but, with the advent of blockchain, interest in this concept has revived, and this is now an active area of research in the blockchain space. Due to the cost saving benefits that smart contracts can bring to the financial services industry by reducing the cost of transactions and simplifying complex contracts, rigorous research is being carried out by various financial and academic institutions in order to formalize and make the implementation of smart contracts easy and practical, as soon as possible.

History

Smart contracts were first theorized by *Nick Szabo* in the late 1990s, but it was almost 20 years before the true potential and benefits of them were truly appreciated. Smart contracts are described by *Szabo* as follows:

“A smart contract is a computerized transaction protocol that executes the terms of a contract. The general objectives are to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries. Related economic goals include lowering fraud loss, arbitrations and enforcement costs, and other transaction costs.”

This idea of smart contracts was implemented in a limited fashion in bitcoin in 2009, where bitcoin transactions can be used to transfer the value between users, over a peer-to-peer network where users do not necessarily trust each other and there is no need for a trusted intermediary.

Definition

There is no consensus on a standard definition of smart contracts. It is essential to define what a smart contract is, and the following is the author's attempt to provide a generalized definition of a smart contract.

A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable.

Dissecting this definition further reveals that a smart contract is in fact a computer program that is written in a language that a computer or target machine can understand. Also, it encompasses agreements between parties in the form of business logic. Another key idea is that smart contracts are automatically executed when certain conditions are met. They are enforceable, which means that all contractual terms are executed as defined and expected, even in the presence of adversaries. Enforcement is a broader term that encompasses traditional enforcement in the form of law, along with implementation of certain measures and controls that make it possible to execute contract terms without requiring any mediation. It should be noted that true smart contracts should not rely on traditional methods of enforcement. Instead, they should work on the principle that *code is law*, meaning that there is no need for an arbitrator or a third party to control or influence the execution of the smart contract. Smart contracts are self-enforcing as opposed to legally enforceable. This might be regarded as a libertarian's dream, but it is entirely possible, and is in line with the true spirit of smart contracts.

Moreover, they are secure and unstoppable, which means that these computer programmes are required to be designed in such a fashion that they are fault tolerant and executable in reasonable amount of time. These programmes should be able to execute and maintain a healthy internal state, even if external factors are unfavorable. For example, imagine a normal computer programme which is encoded with some logic and executes according to the instruction coded within it, but if the environment it is running in or external factors it relies on deviate from the normal or expected state, the programme may react arbitrarily or simply abort. It is important that smart contracts are immune to this type of issue.

Secure and unstoppable may well be considered requirements or desirable features but it will provide greater benefits in the long run if security and unstoppable properties are included in the smart contract definition from the beginning. This will allow researchers to focus on these aspects from the start and will help to build strong foundations on which further research can then be based. There is also a suggestion by some researchers that smart contracts need not be automatically executable; instead they can be what's called *automatable*, due to manual human input required in certain scenarios. Whilst it's true that in some cases human input and control is desirable, it is not absolutely necessary; and, for a contract to be truly *smart*, in the authors opinion, it has to be fully automated. Certain inputs that need to be provided by people can and should also be automated via the use of Oracles. Oracles will be discussed later in this chapter in greater detail.

Smart contracts usually operate by managing their internal state using a state machine model. This allows development of an effective framework for programming smart contracts, where the state of a contract is advanced further based on some predefined criteria and conditions.

There is also on-going debate on the question of whether code is acceptable as a contract in a court of law. This is totally different in presentation from traditional legal prose, albeit they do represent and enforce all contractual clauses but a court of law does not understand code. This raises several questions around how a smart contract can be legally binding: can it be developed in such a way that it is readily acceptable and understandable in a court of law? How can dispute resolution be implemented within the code, and is it possible? Moreover, regulatory and compliance requirements is another topic that needs to be addressed before smart contracts can be used as effectively as traditional legal documents.

The preceding questions open up various possibilities, such as making smart contract code readable not only by machines but also by people. If humans and machines can both understand the code written in a smart contract it might be more acceptable in legal situations, as opposed to just a piece of code that no-one understands except for programmers. This desirable property is an area ripe for research and much research effort has been expended in this area to answer questions around semantics, meaning and interpretation of a contract.

Smart contracts are inherently required to be deterministic in nature. This property will allow a smart contract to be run by any node on a network and achieve the same result. If the result differs even slightly between nodes, consensus then cannot be achieved and a whole paradigm of distributed consensus on blockchain can fail. Moreover, it is also desirable that the contract language itself is deterministic thus ensuring the integrity and stability of the smart contracts. By which I mean, deterministic in the sense that there are no non-deterministic functions used in the language which can produce varied results on different nodes. For example, various floating point operations calculated by various functions in a variety of programming languages can produce different results in different runtime environments. Another example is of some math functions in JavaScript which can produce different results for the same input on different browsers, and which can in turn lead to various bugs. This is highly undesirable in smart contracts because, if results are inconsistent between nodes, then consensus will never be achieved. A deterministic feature ensures that smart contracts always produce the same output for a specific input. In other words, programs once compiled produce a solid and accurate business logic that is completely in line with the requirements programmed in the high level code.

In summary, a smart contract has the following four properties:

- Automatically executable
- Enforceable
- Semantically sound
- Secure and unstoppable.

The first two properties are required as a minimum, whereas the latter two may not be required or implementable in certain scenarios and can be relaxed. For example, a derivatives contract does not perhaps need to be semantically sound and unstoppable but should at least be automatically executable and enforceable at a basic level. On the other hand, a title deed needs to be semantically sound and complete therefore, in order for it to be implemented as a smart contract, the language must be understood by both computers and people. This issue of interpretation was addressed by *Ian Grigg* with his invention of **Ricardian contracts**, which we will look at in more detail in the next section.

Ricardian contracts

Ricardian contracts were originally proposed in the *Financial Cryptography in 7 Layers* paper by Ian Grigg in late 1990s. These contracts were used initially in a bond trading and payment system called **Ricardo**. The key idea is to write a document which is understandable and acceptable by both a court of law and computer software. Ricardian contracts address the challenge of issuance of value over the Internet. It identifies the issuer and captures all the terms and clauses of the contract in a document in order to make it acceptable as a legally binding contract.

Based on the original definition by Ian Grigg at

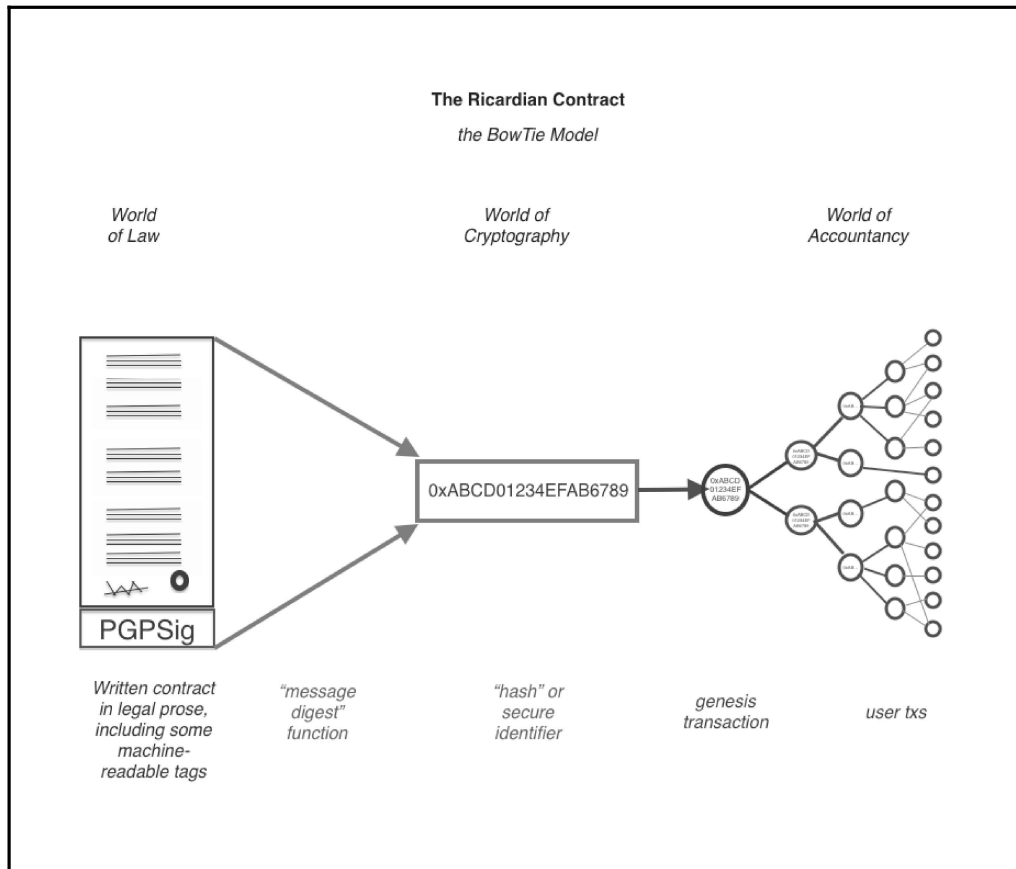
http://iang.org/papers/ricardian_contract.html, a Ricardian contract is a document that has several of the following properties:

- A contract offered by an issuer to holders
- A valuable right held by holders, and managed by the issuer
- Easily readable by people (like a contract on paper)
- Readable by programs (parseable, like a database)
- Digitally signed
- Carries the keys and server information
- Allied with a unique and secure identifier

In practice, the contracts are implemented by producing a single document that contains the terms of the contract in legal prose and the required machine-readable tags. This document is digitally signed by the issuer using their private key. This document is then hashed using a message digest function to produce a hash by which the document can be identified. This hash is then further used and signed by parties during the performance of the contract in order to link each transaction, with the identifier hash thus serving as evidence of intent. This is depicted in the diagram below, usually called a *bowtie* model.

The diagram below shows the **World of Law** on the left hand side from where the document originates. It is then hashed and the resultant message digest is used as an identifier throughout the **World of Accountancy**. The **World of Accountancy** can basically represent any or multiple accounting, trading and information systems that are being used in a business to perform various business operations. The idea behind this flow is that the message digest generated by hashing the document is first used in a so called *genesis transaction*, or first transaction, and then used in every transaction as an identifier throughout the operational execution of the contract.

This way, a secure link is created between the original written contract and every transaction in the *World of Accounting*.



Ricardian contracts, bowtie diagram

A Ricardian contract is different from a smart contract in the sense that a smart contract does not include any contractual document and is focused purely on the execution of the contract. A Ricardian contract, on the other hand, is more concerned with the semantic richness and production of a document that contains contractual legal prose. The semantics of a contract can be divided into two types: operational semantics and denotational semantics. The first type defines the actual execution, correctness and safety of the contract, and the latter is concerned with the real-world meaning of the full contract. Some researchers have differentiated between smart contract code and smart legal contracts where a smart contract is only concerned with the execution of the contract and the second type encompasses both the denotational and operational semantics of a legal agreement. It makes sense to perhaps categorize smart contracts based on the difference between semantics, but it is better to consider smart contracts as a standalone entity that is capable of encoding legal prose and code (business logic) in it.

At bitcoin, a very simple implementation of a smart contract can be observed which is fully oriented towards the execution of the contract, whereas a Ricardian contract is more geared towards producing a document that is understandable by humans, with some parts that a computer program can understand. This can be viewed as legal semantics vs operational performance (semantics vs performance) as shown in the following diagram. This was originally proposed by *Ian Grigg* in his paper *On the intersection of Ricardian and smart contracts*.

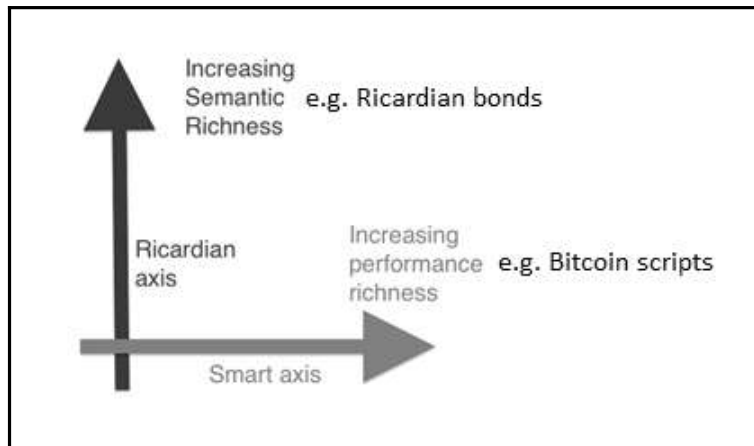


Diagram explaining performance v. semantics are orthogonal issues as described by Ian Grigg; slightly modified to show examples of different types of contracts on both axis

A smart contract is made up to have both of these elements (performance and semantics) embedded together, which completes an ideal model of a smart contract.

A Ricardian contract can be represented as a tuple of three objects, namely *Prose*, *parameters* and *code*. Prose represents the legal contract in regular language; code represents the program that is a computer-understandable representation of legal prose; and parameters join the appropriate parts of the legal contract to the equivalent code.

Ricardian contracts have been implemented in many systems, such as CommonAccord, OpenBazaar, OpenAssets, and Askemos.

Smart contract templates

Smart contracts can be implemented for any industry where required but most current use cases are related to the financial industry. Recent work in smart contract space specific to the financial industry has proposed the idea of smart contract templates. The idea is to build standard templates that provide a framework to support legal agreements for financial instruments. This was proposed by *Clack et al* in their paper named *Smart Contract Templates: Foundations, design landscape and research directions*. The paper also proposed that domain-specific languages should be built in order to support design and implementation of smart contract templates. A language named *CLACK*, a common language for augmented contract knowledge has been proposed and research has begun to develop the language. This language is intended to be very rich and provide a large variety of functions ranging from supporting legal prose to the ability to be executed on multiple platforms and cryptographic functions.

Contracts in the finance industry is not a new concept and various domain-specific language DSLs are already in use in the financial industry to provide specific language for a specific domain. For example, there are DSLs available that support development of insurance products, represent energy derivatives, or are being used to build trading strategies. The list goes on and a comprehensive list of financial domain-specific languages can be found at <http://www.dsifin.org/resources.html>.

It's important to understand the concept of domain-specific languages. These languages are developed with limited expressiveness for a particular application or area of interest.

Domain-specific languages (DSLs) are different from **general-purpose programming languages (GPLs)**: DSLs have a small set of features that are sufficient and optimized for the domain they are intended to be used in and, unlike GPLs, are usually not used to build general purpose large application programmes. Based on the design philosophy of DSLs it can be envisaged that such languages can be developed specifically to write smart contracts. Some work has already been done and Solidity is one such language that has been introduced with Ethereum blockchain to write smart contracts. Serpent is another language that has been introduced with Ethereum even though it's not used as much as Solidity.

This idea of domain-specific languages for smart contract programming can be further extended to a graphical domain-specific language, a smart contract modelling platform where a domain expert (not a programmer) can use a graphical user interface and a canvas to define and draw the semantics and performance of a financial contract. Once the flow has been drawn and completed, it can be emulated first to test and then to deploy from the same system to the target platform, which can be a blockchain. This is also not a new concept and a similar approach is used in the Tibco streambase product, which is a Java based system used for building event-driven high frequency trading systems.

It is proposed that research should also be conducted in the area of developing high level DSLs that can be used to programme a smart contract in a user friendly graphical user interface thus allowing a non-programmer to design a smart contract.

Oracles

Oracles are an important component of the smart contract ecosystem. The limitation with smart contracts is that they cannot access external data which might be required to control the execution of the business logic; for example, the stock price of a security that is required by the contract to release the dividend payments. Oracles can be used to provide external data to smart contracts. An Oracle is an interface that delivers data from an external source to smart contracts. Depending on the industry and requirements, Oracles can deliver different types of data ranging from weather reports, real-world news, and corporate actions to data coming from **Internet of Things (IoT)** devices. Oracles are trusted entities that use a secure channel to transfer data to a smart contract.

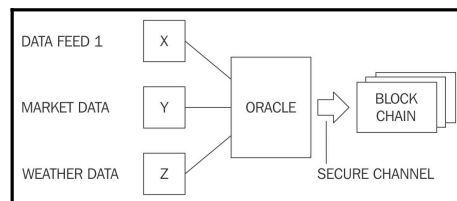
Oracles are also capable of digitally signing the data proving that the source of the data is authentic. Smart contracts can then subscribe to the Oracles, and the smart contracts can either pull the data, or Oracles can push the data to the smart contracts. It is also necessary that Oracles should not be able to manipulate the data they provide and must be able to provide authentic data. Even though Oracles are trusted, it may still be possible in some cases that the data is incorrect due to manipulation. Therefore, it is necessary that Oracles are unable to change the data. This validation can be provided by using various notary schemes, discussed later in the chapter. In this approach an issue can already be seen which perhaps is not desirable in some cases, and that is the issue of trust. How do you trust a third party about the quality and authenticity of data they provide? This is especially true in the financial world, where market data must be accurate and reliable. It might be acceptable for a smart contract designer to accept data for an oracle that is provided by a large reputable trusted third party, but the issue of centralization still remains. These types of Oracles can be called standard or simple Oracles.

Another type of Oracle, which essentially emerged due to the decentralization requirements, can be called *decentralized* Oracles. These types of Oracles can be built based on some distributed mechanism. It can also be envisaged that the Oracles can themselves source data from another blockchain which is driven by distributed consensus, thus ensuring the authenticity of data. For example, one institution running their own private blockchain can publish their data feed via an Oracle that can then be consumed by other blockchains.

Another concept of hardware Oracles is also introduced by researchers where real-world data from physical devices is required. For example, this can be used in telemetry and IoT. However, this approach however requires a mechanism in which hardware devices cannot be tampered with. This can be achieved by using tamper-proof devices.

There are platforms available now to enable a smart contract to get external data using an Oracle. There are different methods used by an Oracle to write data into the blockchain depending on the type of blockchain used. For example in bitcoin blockchain, an oracle can write data to a specific transaction via an `OP_RETURN` Opcode, and a smart contract can monitor that transaction and read the data. Various online services such as <http://www.oraclize.it/> and <https://www.realitykeys.com/> are available that provide oracle services. Also, another service at <https://smartcontract.com/> is available which provides external data and the ability to make payments using smart contracts. The aim of all these services is to enable the smart contract to get the data it needs to execute and make decisions. In order to prove the authenticity of the data retrieved by the Oracles from external sources, mechanisms like TLSnotary can be used which produce proof of communication between the data source and the oracle. This ensures that the data fed back to the smart contract is definitely retrieved from the source. More details about TLSnotary can be found here <https://tlsnotary.org/>.

The following diagram shows a generic model of an oracle and smart contract ecosystem:



A simplified model of an oracle interacting with smart contract on blockchain

Smart Oracles

An idea of Smart Oracle has also been proposed and implemented in *Codius*. Smart Oracles are basically entities just like Oracles, but with the added capability of contract code execution. Smart Oracles proposed by Codius run using Google Native Client, which is a sandboxed environment for running untrusted x86 native code. Codius is available at <https://www.codius.org/>.

Deploying smart contracts on a blockchain

Smart contracts may or may not be deployed on a blockchain but it makes sense to deploy them on a blockchain due to the distributed consensus mechanism provided by blockchain. Ethereum is an example of a blockchain that natively supports the development and deployment of smart contracts. Smart contracts on Ethereum blockchain are usually part of a larger application such as **Decentralized Autonomous organization (DAOs)**.

As a comparison, in bitcoin blockchain the `lock_time` field in the bitcoin transaction can be seen as an enabler of a basic version of a smart contract. The `lock_time` field enables a transaction to be locked until a specified time or after a number of blocks, thus enforcing a basic contract that a certain transaction can only be unlocked if certain conditions (elapsed time or number of blocks) is met. However, this is very limited in nature and should be only viewed as an example of a basic smart contract. In addition to the above mentioned example, bitcoin scripting language, though limited, can be used to construct basic smart contracts. One possibility is to fund a bitcoin address that can be spent by anyone who demonstrates a hash collision attack. This idea was presented on the Bitcointalk forum and more information can be found at <https://bitcointalk.org/index.php?topic=293382.0>. This can also be considered a basic form of smart contract.

The DAO

The DAO is one of the highest crowdfunded projects, and started in April 2016. This was basically a set of smart contracts written in order to provide a platform for investment. Due to a bug in the code this was hacked in June 2016 and an equivalent of 50 million dollars was siphoned out of the DAO into another account. This resulted in a hard fork on Ethereum in order to recover from the attack. It should be noted that the notion of *code is law*, or unstoppable smart contracts, should be viewed with some scepticism as the implementation of these concepts is not mature enough to warrant full and unquestionable trust. This is evident from the recent events where the Ethereum foundation was able to stop and change the execution of *The DAO* by introducing a hard fork. Though this hard fork was introduced for genuine reasons, it goes against the true spirit of decentralization and the notion of *code is law*. On the other hand, resistance against this hard fork and some miners who decided to keep mining on the original chain resulted in the creation of Ethereum Classic. This is the original, non-forked Ethereum blockchain where *code is still law*.

This attack highlights the dangers of smart contracts and the absolute need to develop a formal language for smart contracts. The attack also highlighted the importance of thorough testing. There have been various vulnerabilities discovered in Ethereum recently around the smart contract development language. Therefore it is of utmost importance that a standard framework is developed to address all these issues. Some work has already begun as discussed previously, but this area is ripe for more research in order to address limitations in smart contract languages.

Summary

This chapter started by introducing a history of smart contracts, and was followed by a detailed discussion on the definition of a smart contract. As there is no agreement on the standard definition of a smart contract, we attempted to introduce a definition that encompasses the crux of smart contracts. An introduction to Ricardian contracts was also provided, and the difference between Ricardian contracts and smart contracts was explained, highlighting the fact that Ricardian contracts are concerned with the definition of the contract whereas smart contracts are geared towards the actual execution of the contract. The concept of smart contract templates was discussed, on the subject of which high quality active research is currently being conducted in academia and industry. Some ideas about the possibility of creating high level domain-specific languages were also discussed to create smart contracts or smart contract templates. In later sections of the chapter, the concepts of Oracles was introduced followed by a brief discussion on the DAO, and security issues in DAO and smart contracts.